

Lecture 07 - Visibility Determination

This lecture asks which surfaces are visible from a viewpoint. Rasterization can generate many fragment candidates, but visibility decides which ones should affect the image.

Source chunk	course_text_parts/03_lectures/07_visibility-determination.txt
Extracted slide pages	69
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Per-slide commentary

Each extracted slide page gets its own note. Use this section as the spoken commentary that the terse slide deck is missing.

Page 1	Untitled slide
Source cue	The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image.
Professor-style explanation	For 'Untitled slide', I would not just read the bullet points aloud. I would ask what problem the slide is solving and how it connects to the previous and next stage. The slide gives us this anchor: The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image. Turn the slide into a causal explanation: this input is processed by this idea, which produces this result, and that result matters later.
Technical commentary	This slide is about Untitled slide. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Untitled slide' into a causal sentence instead of repeating the slide title?
Page 2	Visibility Determination
Source cue	- Problem / - Given scene objects and virtual camera / - Determine visible parts of scene objects / - Not occluded by same scene object / - Not occluded by other scene objects

Professor-style explanation	This slide should be read as camera geometry. With 'Visibility Determination', the question is how a 3D view becomes coordinates that can be clipped, divided, mapped to the viewport, and rasterized. The slide gives us this anchor: - Problem / - Given scene objects and virtual camera / - Determine visible parts of scene objects / - Not occluded by same scene object / - Not occluded by other scene objects. Keep separate the camera/view transform, the projection matrix, the perspective divide, and the final viewport transform; many mistakes come from blending these steps together.
Technical commentary	This slide is about Visibility Determination. Read it as camera geometry. Track how 3D view-space positions become clip coordinates, normalized device coordinates, and finally screen locations. The visible cue is: - Problem / - Given scene objects and virtual camera / - Determine visible parts of scene objects / - Not occluded by same scene object / - Not occluded by other scene objects
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Visibility Determination' changes positions before rasterization?

Page 3	Object-Based Algorithms
Source cue	- Geometry-based analysis of scene objects before rasterization / - Compare each scene object against all other scene objects / - Determine visible parts based on virtual camera / - Rasterize the visible scene parts / For each object in sceneObjects
Professor-style explanation	This slide should be read as camera geometry. With 'Object-Based Algorithms', the question is how a 3D view becomes coordinates that can be clipped, divided, mapped to the viewport, and rasterized. The slide gives us this anchor: - Geometry-based analysis of scene objects before rasterization / - Compare each scene object against all other scene objects / - Determine visible parts based on virtual camera / - Rasterize the visible scene parts / For each object in sceneObjects. Keep separate the camera/view transform, the projection matrix, the perspective divide, and the final viewport transform; many mistakes come from blending these steps together.
Technical commentary	This slide is about Object-Based Algorithms. Read it as camera geometry. Track how 3D view-space positions become clip coordinates, normalized device coordinates, and finally screen locations. The visible cue is: - Geometry-based analysis of scene objects before rasterization / - Compare each scene object against all other scene objects / - Determine visible parts based on virtual camera / - Rasterize the visible scene parts / For each object in sceneObjects
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Object-Based Algorithms' changes positions before rasterization?

Page 4	Image-Based Algorithms
--------	------------------------

Source cue	- Pixel-based analysis of scene objects after rasterization / - Rasterize scene objects based on image resolution / - Investigate for each pixel which scene object is visible / For each pixel in outputRaster / // Determine which scene object is visible at this pixel
Professor-style explanation	Here the lecture moves from continuous geometry to a discrete grid. 'Image-Based Algorithms' asks which pixels or samples are covered by an ideal mathematical primitive. The slide gives us this anchor: - Pixel-based analysis of scene objects after rasterization / - Rasterize scene objects based on image resolution / - Investigate for each pixel which scene object is visible / For each pixel in outputRaster / // Determine which scene object is visible at this pixel. The key spoken explanation is: rasterization creates fragment candidates and interpolated values, but it does not by itself guarantee that a fragment becomes the final visible pixel.
Technical commentary	This slide is about Image-Based Algorithms. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: - Pixel-based analysis of scene objects after rasterization / - Rasterize scene objects based on image resolution / - Investigate for each pixel which scene object is visible / For each pixel in outputRaster / // Determine which scene object is visible at this pixel
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Image-Based Algorithms'?
Page 5	7.1 Object-Based Algorithms
Source cue	7.2 Binary Space Partitioning / 7.3 Warnock Algorithm / 7.4 Depth Buffer Algorithm / 7.5 Depth Buffer Extensions / 7.6 Ray Casting
Professor-style explanation	On this slide, I would emphasize that OpenGL is a controlled state machine around the GPU pipeline. '7.1 Object-Based Algorithms' is not about one magic render call; it is about objects, bindings, shader interfaces, buffers, and state being consistent at draw time. The slide gives us this anchor: 7.2 Binary Space Partitioning / 7.3 Warnock Algorithm / 7.4 Depth Buffer Algorithm / 7.5 Depth Buffer Extensions / 7.6 Ray Casting. If the output is wrong, you inspect which object owns the data, which shader consumes it, and which state changes the result.
Technical commentary	This slide is about 7.1 Object-Based Algorithms. Read it as concrete API state and GPU data movement. Ask which object is bound, which shader stage consumes it, and which state affects the draw call. The visible cue is: 7.2 Binary Space Partitioning / 7.3 Warnock Algorithm / 7.4 Depth Buffer Algorithm / 7.5 Depth Buffer Extensions / 7.6 Ray Casting
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in '7.1 Object-Based Algorithms'?
Page 6	7.1 Object-Based Algorithms
Source cue	Exploiting sorting and clipping

Professor-style explanation	For '7.1 Object-Based Algorithms', think of a boundary test. The renderer does not want arbitrary geometry continuing forever; it needs to decide what part of a primitive is inside the valid region. The slide gives us this anchor: Exploiting sorting and clipping. A good explanation says which object is tested, which boundary is used, whether the primitive is accepted, rejected, or cut, and where new intersection points may appear.
Technical commentary	This slide is about 7.1 Object-Based Algorithms. Read it as boundary logic. Identify what is inside, what is outside, what can be trivially accepted/rejected, and where intersections are created. The visible cue is: Exploiting sorting and clipping
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in '7.1 Object-Based Algorithms'?

Page 7	Painter's Algorithm 1/3
Source cue	Bob Ross – Grandeur of Summer
Professor-style explanation	For 'Painter's Algorithm 1/3', I would not just read the bullet points aloud. I would ask what problem the slide is solving and how it connects to the previous and next stage. The slide gives us this anchor: Bob Ross – Grandeur of Summer. Turn the slide into a causal explanation: this input is processed by this idea, which produces this result, and that result matters later.
Technical commentary	This slide is about Painter's Algorithm 1/3. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: Bob Ross – Grandeur of Summer
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Painter's Algorithm 1/3' into a causal sentence instead of repeating the slide title?

Page 8	Painter's Algorithm 2/3
Source cue	- First algorithm for object-based visible polygon determination / - Input: set of polygons / - Output: rendering of visible polygons / - Procedure: simulate drawing process of a painting / 1. Sort polygons into list in back-to-front order
Professor-style explanation	For 'Painter's Algorithm 2/3', I would not just read the bullet points aloud. I would ask what problem the slide is solving and how it connects to the previous and next stage. The slide gives us this anchor: - First algorithm for object-based visible polygon determination / - Input: set of polygons / - Output: rendering of visible polygons / - Procedure: simulate drawing process of a painting / 1. Sort polygons into list in back-to-front order. Turn the slide into a causal explanation: this input is processed by this idea, which produces this result, and that result matters later.

Technical commentary	This slide is about Painter's Algorithm 2/3. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - First algorithm for object-based visible polygon determination / - Input: set of polygons / - Output: rendering of visible polygons / - Procedure: simulate drawing process of a painting / 1. Sort polygons into list in back-to-front order
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Painter's Algorithm 2/3' into a causal sentence instead of repeating the slide title?

Page 9	Painter's Algorithm 3/3
Source cue	- Advantages / - Simple implementation / - No special graphics hardware necessary / - Disadvantages / - Works for polygons only instead of 3D scene objects
Professor-style explanation	For 'Painter's Algorithm 3/3', I would not just read the bullet points aloud. I would ask what problem the slide is solving and how it connects to the previous and next stage. The slide gives us this anchor: - Advantages / - Simple implementation / - No special graphics hardware necessary / - Disadvantages / - Works for polygons only instead of 3D scene objects. Turn the slide into a causal explanation: this input is processed by this idea, which produces this result, and that result matters later.
Technical commentary	This slide is about Painter's Algorithm 3/3. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - Advantages / - Simple implementation / - No special graphics hardware necessary / - Disadvantages / - Works for polygons only instead of 3D scene objects
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Painter's Algorithm 3/3' into a causal sentence instead of repeating the slide title?

Page 10	Weiler-Atherton Algorithm 1/3
Source cue	- Clipping-based divide-and-conquer approach / - Input: set of polygons / - Output: set of visible polygon parts / - Procedure / 1. Sort polygons into list ■ in front-to-back order
Professor-style explanation	For 'Weiler-Atherton Algorithm 1/3', think of a boundary test. The renderer does not want arbitrary geometry continuing forever; it needs to decide what part of a primitive is inside the valid region. The slide gives us this anchor: - Clipping-based divide-and-conquer approach / - Input: set of polygons / - Output: set of visible polygon parts / - Procedure / 1. Sort polygons into list ■ in front-to-back order. A good explanation says which object is tested, which boundary is used, whether the primitive is accepted, rejected, or cut, and where new intersection points may appear.

Technical commentary	This slide is about Weiler-Atherton Algorithm 1/3. Read it as boundary logic. Identify what is inside, what is outside, what can be trivially accepted/rejected, and where intersections are created. The visible cue is: - Clipping-based divide-and-conquer approach / - Input: set of polygons / - Output: set of visible polygon parts / - Procedure / 1. Sort polygons into list ■ in front-to-back order
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Weiler-Atherton Algorithm 1/3'?
Page 11	Weiler-Atherton Algorithm 2/3
Source cue	- Example from original paper / - Z value order flipped / - Bold: clip polygon / - Gray: unoccluded parts
Professor-style explanation	For 'Weiler-Atherton Algorithm 2/3', think of a boundary test. The renderer does not want arbitrary geometry continuing forever; it needs to decide what part of a primitive is inside the valid region. The slide gives us this anchor: - Example from original paper / - Z value order flipped / - Bold: clip polygon / - Gray: unoccluded parts. A good explanation says which object is tested, which boundary is used, whether the primitive is accepted, rejected, or cut, and where new intersection points may appear.
Technical commentary	This slide is about Weiler-Atherton Algorithm 2/3. Read it as boundary logic. Identify what is inside, what is outside, what can be trivially accepted/rejected, and where intersections are created. The visible cue is: - Example from original paper / - Z value order flipped / - Bold: clip polygon / - Gray: unoccluded parts
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Weiler-Atherton Algorithm 2/3'?
Page 12	Weiler-Atherton Algorithm 3/3
Source cue	- Advantages / - Can handle lines and polygons / - Exact calculation of the visible polygons / - Can be used for wireframe rendering / with different edge styles
Professor-style explanation	For 'Weiler-Atherton Algorithm 3/3', think of a boundary test. The renderer does not want arbitrary geometry continuing forever; it needs to decide what part of a primitive is inside the valid region. The slide gives us this anchor: - Advantages / - Can handle lines and polygons / - Exact calculation of the visible polygons / - Can be used for wireframe rendering / with different edge styles. A good explanation says which object is tested, which boundary is used, whether the primitive is accepted, rejected, or cut, and where new intersection points may appear.
Technical commentary	This slide is about Weiler-Atherton Algorithm 3/3. Read it as boundary logic. Identify what is inside, what is outside, what can be trivially accepted/rejected, and where intersections are created. The visible cue is: - Advantages / - Can handle lines and polygons / - Exact calculation of the visible polygons / - Can be used for wireframe rendering / with different edge styles

Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Weiler-Atherton Algorithm 3/3'?
Page 13	Back Face Culling 1/2
Source cue	- Solves visibility determination for one convex, opaque scene object, / as its back faces are not visible / - Algorithm culls away polygons facing away from virtual camera / - Input: convex, opaque 3D scene object / - Output: set of front-facing polygons of input scene object
Professor-style explanation	This slide should be read as camera geometry. With 'Back Face Culling 1/2', the question is how a 3D view becomes coordinates that can be clipped, divided, mapped to the viewport, and rasterized. The slide gives us this anchor: - Solves visibility determination for one convex, opaque scene object, / as its back faces are not visible / - Algorithm culls away polygons facing away from virtual camera / - Input: convex, opaque 3D scene object / - Output: set of front-facing polygons of input scene object. Keep separate the camera/view transform, the projection matrix, the perspective divide, and the final viewport transform; many mistakes come from blending these steps together.
Technical commentary	This slide is about Back Face Culling 1/2. Read it as camera geometry. Track how 3D view-space positions become clip coordinates, normalized device coordinates, and finally screen locations. The visible cue is: - Solves visibility determination for one convex, opaque scene object, / as its back faces are not visible / - Algorithm culls away polygons facing away from virtual camera / - Input: convex, opaque 3D scene object / - Output: set of front-facing polygons of input scene object
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Back Face Culling 1/2' changes positions before rasterization?
Page 14	Back Face Culling 2/2
Source cue	- Advantages / - Simple implementation / - An average of 50% of the areas of a scene can be eliminated in this way / - Reduces number of polygons to be processed, / thus reduces GPU transfers and speeds up the rendering process
Professor-style explanation	For 'Back Face Culling 2/2', imagine following one piece of scene data through the renderer. It starts as model or application data, then passes through transformations, primitive processing, rasterization, fragment processing, tests, and finally the framebuffer. The slide gives us this anchor: - Advantages / - Simple implementation / - An average of 50% of the areas of a scene can be eliminated in this way / - Reduces number of polygons to be processed, / thus reduces GPU transfers and speeds up the rendering process. The important point is that each stage changes the representation, so debugging means asking where the representation first became wrong.

Technical commentary	This slide is about Back Face Culling 2/2. Read it as a data-flow explanation: scene or model data is processed step by step until valid framebuffer updates remain. The visible cue is: - Advantages / - Simple implementation / - An average of 50% of the areas of a scene can be eliminated in this way / - Reduces number of polygons to be processed, / thus reduces GPU transfers and speeds up the rendering process
Why it matters	Pipeline understanding lets you localize rendering errors instead of guessing randomly.
Check yourself	Can you name the input and output representation for 'Back Face Culling 2/2' in the rendering pipeline?
Page 15	Robert's Algorithm
Source cue	- First algorithm for object-based visible line determination / - Input: set of convex 3D scene objects / - Output: set of visible lines / - Procedure: perform for all scene objects the following steps / 1. Remove self-occluded surfaces through back-face culling
Professor-style explanation	Here the lecture moves from continuous geometry to a discrete grid. 'Robert's Algorithm' asks which pixels or samples are covered by an ideal mathematical primitive. The slide gives us this anchor: - First algorithm for object-based visible line determination / - Input: set of convex 3D scene objects / - Output: set of visible lines / - Procedure: perform for all scene objects the following steps / 1. Remove self-occluded surfaces through back-face culling. The key spoken explanation is: rasterization creates fragment candidates and interpolated values, but it does not by itself guarantee that a fragment becomes the final visible pixel.
Technical commentary	This slide is about Robert's Algorithm. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: - First algorithm for object-based visible line determination / - Input: set of convex 3D scene objects / - Output: set of visible lines / - Procedure: perform for all scene objects the following steps / 1. Remove self-occluded surfaces through back-face culling
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Robert's Algorithm'?
Page 16	Assessment Robert's Algorithm
Source cue	- Advantages / - Determination of visible lines was particularly important / for presentation on vector screens / - Today used in non-photorealistic rendering / to generate line drawings

Professor-style explanation	Here the lecture moves from continuous geometry to a discrete grid. 'Assessment Robert's Algorithm' asks which pixels or samples are covered by an ideal mathematical primitive. The slide gives us this anchor: - Advantages / - Determination of visible lines was particularly important / for presentation on vector screens / - Today used in non-photorealistic rendering / to generate line drawings. The key spoken explanation is: rasterization creates fragment candidates and interpolated values, but it does not by itself guarantee that a fragment becomes the final visible pixel.
Technical commentary	This slide is about Assessment Robert's Algorithm. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: - Advantages / - Determination of visible lines was particularly important / for presentation on vector screens / - Today used in non-photorealistic rendering / to generate line drawings
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Assessment Robert's Algorithm'?

Page 17	7.2 Binary Space Partitioning
Source cue	Hierarchical scene bisection
Professor-style explanation	For '7.2 Binary Space Partitioning', I would not just read the bullet points aloud. I would ask what problem the slide is solving and how it connects to the previous and next stage. The slide gives us this anchor: Hierarchical scene bisection. Turn the slide into a causal explanation: this input is processed by this idea, which produces this result, and that result matters later.
Technical commentary	This slide is about 7.2 Binary Space Partitioning. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: Hierarchical scene bisection
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn '7.2 Binary Space Partitioning' into a causal sentence instead of repeating the slide title?

Page 18	Spatial Data Structures
Source cue	- Spatial data structures are specialized / for organizing and storing scene data / - Types of spatial data structures / - Quad Trees: Optimize 2D data retrieval by dividing space it into four nodes / - Octrees: Optimize 3D data retrieval by dividing space it into eight nodes

Professor-style explanation	For 'Spatial Data Structures', I would not just read the bullet points aloud. I would ask what problem the slide is solving and how it connects to the previous and next stage. The slide gives us this anchor: - Spatial data structures are specialized / for organizing and storing scene data / - Types of spatial data structures / - Quad Trees: Optimize 2D data retrieval by dividing space it into four nodes / - Octrees: Optimize 3D data retrieval by dividing space it into eight nodes. Turn the slide into a causal explanation: this input is processed by this idea, which produces this result, and that result matters later.
Technical commentary	This slide is about Spatial Data Structures. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - Spatial data structures are specialized / for organizing and storing scene data / - Types of spatial data structures / - Quad Trees: Optimize 2D data retrieval by dividing space it into four nodes / - Octrees: Optimize 3D data retrieval by dividing space it into eight nodes
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Spatial Data Structures' into a causal sentence instead of repeating the slide title?

Page 19	BSP Trees 1/2
Source cue	- BSP Trees are Binary Space Partitioning trees / that recursively subdivide space into convex subsets through hyperplanes / - Key functions / - Space organization: divide 3D space into subspace represented as nodes / - Query efficiency: facilitate rapid querying like determining object visibility
Professor-style explanation	With 'BSP Trees 1/2', the question is no longer just whether geometry exists, but whether it is visible from a viewpoint. The slide gives us this anchor: - BSP Trees are Binary Space Partitioning trees / that recursively subdivide space into convex subsets through hyperplanes / - Key functions / - Space organization: divide 3D space into subspace represented as nodes / - Query efficiency: facilitate rapid querying like determining object visibility. Explain the method by naming its decision space: does it compare objects, split image regions, cast rays, or compare per-fragment depth values? That tells you what it can handle well.
Technical commentary	This slide is about BSP Trees 1/2. Read it as an occlusion decision. Decide whether the method reasons about objects, image regions, rays, or per-fragment depth comparisons. The visible cue is: - BSP Trees are Binary Space Partitioning trees / that recursively subdivide space into convex subsets through hyperplanes / - Key functions / - Space organization: divide 3D space into subspace represented as nodes / - Query efficiency: facilitate rapid querying like determining object visibility
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'BSP Trees 1/2' works per object, per image region, per ray, or per fragment?

Page 20	BSP Trees 2/2
---------	---------------

Source cue	- Definition binary tree / - A binary tree is a tree with at most two child nodes per node / - Different traversal schemes for binary trees exist A X X / - In-order traversal: first left child, then root, then right child / B C Y C
Professor-style explanation	With 'BSP Trees 2/2', the question is no longer just whether geometry exists, but whether it is visible from a viewpoint. The slide gives us this anchor: - Definition binary tree / - A binary tree is a tree with at most two child nodes per node / - Different traversal schemes for binary trees exist A X X / - In-order traversal: first left child, then root, then right child / B C Y C. Explain the method by naming its decision space: does it compare objects, split image regions, cast rays, or compare per-fragment depth values? That tells you what it can handle well.
Technical commentary	This slide is about BSP Trees 2/2. Read it as an occlusion decision. Decide whether the method reasons about objects, image regions, rays, or per-fragment depth comparisons. The visible cue is: - Definition binary tree / - A binary tree is a tree with at most two child nodes per node / - Different traversal schemes for binary trees exist A X X / - In-order traversal: first left child, then root, then right child / B C Y C
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'BSP Trees 2/2' works per object, per image region, per ray, or per fragment?

Page 21	Node Data Structure
Source cue	- Each node contains / - Hyperplane / - List of scene geometry associated with the node / Class BSPNode: / Declare plane as Plane
Professor-style explanation	With 'Node Data Structure', the question is no longer just whether geometry exists, but whether it is visible from a viewpoint. The slide gives us this anchor: - Each node contains / - Hyperplane / - List of scene geometry associated with the node / Class BSPNode: / Declare plane as Plane. Explain the method by naming its decision space: does it compare objects, split image regions, cast rays, or compare per-fragment depth values? That tells you what it can handle well.
Technical commentary	This slide is about Node Data Structure. Read it as an occlusion decision. Decide whether the method reasons about objects, image regions, rays, or per-fragment depth comparisons. The visible cue is: - Each node contains / - Hyperplane / - List of scene geometry associated with the node / Class BSPNode: / Declare plane as Plane
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Node Data Structure' works per object, per image region, per ray, or per fragment?

Page 22	BSP Tree Construction
Source cue	- Choose a hyperplane from scene geometry for root node / and associate that plane with root node, split remaining geometry at plane / - Proceed recursively with the two resulting sets of scene geometries / and check each polygon against hyperplane / - Coinciding: associate with node

Professor-style explanation	With 'BSP Tree Construction', the question is no longer just whether geometry exists, but whether it is visible from a viewpoint. The slide gives us this anchor: - Choose a hyperplane from scene geometry for root node / and associate that plane with root node, split remaining geometry at plane / - Proceed recursively with the two resulting sets of scene geometries / and check each polygon against hyperplane / - Coinciding: associate with node. Explain the method by naming its decision space: does it compare objects, split image regions, cast rays, or compare per-fragment depth values? That tells you what it can handle well.
Technical commentary	This slide is about BSP Tree Construction. Read it as an occlusion decision. Decide whether the method reasons about objects, image regions, rays, or per-fragment depth comparisons. The visible cue is: - Choose a hyperplane from scene geometry for root node / and associate that plane with root node, split remaining geometry at plane / - Proceed recursively with the two resulting sets of scene geometries / and check each polygon against hyperplane / - Coinciding: associate with node
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'BSP Tree Construction' works per object, per image region, per ray, or per fragment?

Page 23	BSP Tree Construction - Example
Source cue	1 2 / 3 4 / A B E / 3 1
Professor-style explanation	With 'BSP Tree Construction - Example', the question is no longer just whether geometry exists, but whether it is visible from a viewpoint. The slide gives us this anchor: 1 2 / 3 4 / A B E / 3 1. Explain the method by naming its decision space: does it compare objects, split image regions, cast rays, or compare per-fragment depth values? That tells you what it can handle well.
Technical commentary	This slide is about BSP Tree Construction - Example. Read it as an occlusion decision. Decide whether the method reasons about objects, image regions, rays, or per-fragment depth comparisons. The visible cue is: 1 2 / 3 4 / A B E / 3 1
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'BSP Tree Construction - Example' works per object, per image region, per ray, or per fragment?

Page 24	BSP Tree Construction – Pseudo Code
Source cue	Function buildBSPTree(BSPNode node, List of Polygons list) // Select one polygon to use as a hyperplane / Polygon p = Get first polygon from list / Set node.plane to the plane of polygon p / Add polygon p to node.geometry

Professor-style explanation	With 'BSP Tree Construction – Pseudo Code', the question is no longer just whether geometry exists, but whether it is visible from a viewpoint. The slide gives us this anchor: Function buildBSPTree(BSPNode node, List of Polygons list) // Select one polygon to use as a hyperplane / Polygon p = Get first polygon from list / Set node.plane to the plane of polygon p / Add polygon p to node.geometry. Explain the method by naming its decision space: does it compare objects, split image regions, cast rays, or compare per-fragment depth values? That tells you what it can handle well.
Technical commentary	This slide is about BSP Tree Construction – Pseudo Code. Read it as an occlusion decision. Decide whether the method reasons about objects, image regions, rays, or per-fragment depth comparisons. The visible cue is: Function buildBSPTree(BSPNode node, List of Polygons list) // Select one polygon to use as a hyperplane / Polygon p = Get first polygon from list / Set node.plane to the plane of polygon p / Add polygon p to node.geometry
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'BSP Tree Construction – Pseudo Code' works per object, per image region, per ray, or per fragment?

Page 25	Visibility Determination
Source cue	- Visibility determination through modified depth-first traversal / respecting the current camera / - Tree traversal handles the entire scene geometry / - Drawing is done in analogy to painter's algorithm / - Traversing provides the necessary "strict back-to-front order"
Professor-style explanation	This slide should be read as camera geometry. With 'Visibility Determination', the question is how a 3D view becomes coordinates that can be clipped, divided, mapped to the viewport, and rasterized. The slide gives us this anchor: - Visibility determination through modified depth-first traversal / respecting the current camera / - Tree traversal handles the entire scene geometry / - Drawing is done in analogy to painter's algorithm / - Traversing provides the necessary "strict back-to-front order". Keep separate the camera/view transform, the projection matrix, the perspective divide, and the final viewport transform; many mistakes come from blending these steps together.
Technical commentary	This slide is about Visibility Determination. Read it as camera geometry. Track how 3D view-space positions become clip coordinates, normalized device coordinates, and finally screen locations. The visible cue is: - Visibility determination through modified depth-first traversal / respecting the current camera / - Tree traversal handles the entire scene geometry / - Drawing is done in analogy to painter's algorithm / - Traversing provides the necessary "strict back-to-front order"
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Visibility Determination' changes positions before rasterization?

Page 26	Visibility Determination – Pseudo Code
---------	--

Source cue	Function BSPHiddenSurfaceTraversal(BSPNode node, Point eye) // Calculate the signed distance from the eye point to the partition plane / float distance = classifyPoint(node.plane, eye) / If distance < 0 Then // Eye-point is on the back side of the plane
Professor-style explanation	With 'Visibility Determination – Pseudo Code', the question is no longer just whether geometry exists, but whether it is visible from a viewpoint. The slide gives us this anchor: Function BSPHiddenSurfaceTraversal(BSPNode node, Point eye) // Calculate the signed distance from the eye point to the partition plane / float distance = classifyPoint(node.plane, eye) / If distance < 0 Then // Eye-point is on the back side of the plane. Explain the method by naming its decision space: does it compare objects, split image regions, cast rays, or compare per-fragment depth values? That tells you what it can handle well.
Technical commentary	This slide is about Visibility Determination – Pseudo Code. Read it as an occlusion decision. Decide whether the method reasons about objects, image regions, rays, or per-fragment depth comparisons. The visible cue is: Function BSPHiddenSurfaceTraversal(BSPNode node, Point eye) // Calculate the signed distance from the eye point to the partition plane / float distance = classifyPoint(node.plane, eye) / If distance < 0 Then // Eye-point is on the back side of the plane
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Visibility Determination – Pseudo Code' works per object, per image region, per ray, or per fragment?
Page 27	BSP Tree Traversal - Example
Source cue	■ ■ / 4 1 2 / ■ ■ ■ ■ / 3 4 / ■ ■ ■ ■
Professor-style explanation	With 'BSP Tree Traversal - Example', the question is no longer just whether geometry exists, but whether it is visible from a viewpoint. The slide gives us this anchor: ■ ■ / 4 1 2 / ■ ■ ■ ■ / 3 4 / ■ ■ ■ ■. Explain the method by naming its decision space: does it compare objects, split image regions, cast rays, or compare per-fragment depth values? That tells you what it can handle well.
Technical commentary	This slide is about BSP Tree Traversal - Example. Read it as an occlusion decision. Decide whether the method reasons about objects, image regions, rays, or per-fragment depth comparisons. The visible cue is: ■ ■ / 4 1 2 / ■ ■ ■ ■ / 3 4 / ■ ■ ■ ■
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'BSP Tree Traversal - Example' works per object, per image region, per ray, or per fragment?
Page 28	Splitting Planes Selection
Source cue	- Hyperplane selection influences tree balance and required geometry splits / - Two alternative approaches exist / - Planes can be derived from scene geometry / - Planes can be chosen freely / - Useful heuristics

Professor-style explanation	For 'Splitting Planes Selection', I would not just read the bullet points aloud. I would ask what problem the slide is solving and how it connects to the previous and next stage. The slide gives us this anchor: - Hyperplane selection influences tree balance and required geometry splits / - Two alternative approaches exist / - Planes can be derived from scene geometry / - Planes can be chosen freely / - Useful heuristics. Turn the slide into a causal explanation: this input is processed by this idea, which produces this result, and that result matters later.
Technical commentary	This slide is about Splitting Planes Selection. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - Hyperplane selection influences tree balance and required geometry splits / - Two alternative approaches exist / - Planes can be derived from scene geometry / - Planes can be chosen freely / - Useful heuristics
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Splitting Planes Selection' into a causal sentence instead of repeating the slide title?

Page 29	Traversal Optimization
Source cue	- Sub-trees can be skipped during traversal if content is outside view frustum / - Testing is done by checking 8 corner points of 3D view frustum / - If there are two points on different sides, then the subtree is traversed / - If all points are in one subspace, then the other subspace does not have / to be traversed (view-frustum culling)
Professor-style explanation	With 'Traversal Optimization', the question is no longer just whether geometry exists, but whether it is visible from a viewpoint. The slide gives us this anchor: - Sub-trees can be skipped during traversal if content is outside view frustum / - Testing is done by checking 8 corner points of 3D view frustum / - If there are two points on different sides, then the subtree is traversed / - If all points are in one subspace, then the other subspace does not have / to be traversed (view-frustum culling). Explain the method by naming its decision space: does it compare objects, split image regions, cast rays, or compare per-fragment depth values? That tells you what it can handle well.
Technical commentary	This slide is about Traversal Optimization. Read it as an occlusion decision. Decide whether the method reasons about objects, image regions, rays, or per-fragment depth comparisons. The visible cue is: - Sub-trees can be skipped during traversal if content is outside view frustum / - Testing is done by checking 8 corner points of 3D view frustum / - If there are two points on different sides, then the subtree is traversed / - If all points are in one subspace, then the other subspace does not have / to be traversed (view-frustum culling)
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Traversal Optimization' works per object, per image region, per ray, or per fragment?

Page 30	Assessment BSP Trees
---------	----------------------

Source cue	- Advantages / - Simple implementation / - Do not need a depth buffer / - Applicable to static and dynamic scenes / (static scene geometry precalculated, dynamic objects inserted)
Professor-style explanation	On this slide, I would emphasize that OpenGL is a controlled state machine around the GPU pipeline. 'Assessment BSP Trees' is not about one magic render call; it is about objects, bindings, shader interfaces, buffers, and state being consistent at draw time. The slide gives us this anchor: - Advantages / - Simple implementation / - Do not need a depth buffer / - Applicable to static and dynamic scenes / (static scene geometry precalculated, dynamic objects inserted). If the output is wrong, you inspect which object owns the data, which shader consumes it, and which state changes the result.
Technical commentary	This slide is about Assessment BSP Trees. Read it as concrete API state and GPU data movement. Ask which object is bound, which shader stage consumes it, and which state affects the draw call. The visible cue is: - Advantages / - Simple implementation / - Do not need a depth buffer / - Applicable to static and dynamic scenes / (static scene geometry precalculated, dynamic objects inserted)
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Assessment BSP Trees'?
Page 31	7.3 Warnock Algorithm
Source cue	Image-based divide-and-conquer strategy
Professor-style explanation	With '7.3 Warnock Algorithm', the question is no longer just whether geometry exists, but whether it is visible from a viewpoint. The slide gives us this anchor: Image-based divide-and-conquer strategy. Explain the method by naming its decision space: does it compare objects, split image regions, cast rays, or compare per-fragment depth values? That tells you what it can handle well.
Technical commentary	This slide is about 7.3 Warnock Algorithm. Read it as an occlusion decision. Decide whether the method reasons about objects, image regions, rays, or per-fragment depth comparisons. The visible cue is: Image-based divide-and-conquer strategy
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether '7.3 Warnock Algorithm' works per object, per image region, per ray, or per fragment?
Page 32	Warnock Algorithm
Source cue	- Hybrid object- and image-based divide-and-conquer approach / - Input: set of polygons and viewport / - Output: set of visible polygon parts / - Procedure: exploit spatial coherence of projected polygons to divide viewport / - Divide viewport recursively into squared regions

Professor-style explanation	This slide should be read as camera geometry. With 'Warnock Algorithm', the question is how a 3D view becomes coordinates that can be clipped, divided, mapped to the viewport, and rasterized. The slide gives us this anchor: - Hybrid object- and image-based divide-and-conquer approach / - Input: set of polygons and viewport / - Output: set of visible polygon parts / - Procedure: exploit spatial coherence of projected polygons to divide viewport / - Divide viewport recursively into squared regions. Keep separate the camera/view transform, the projection matrix, the perspective divide, and the final viewport transform; many mistakes come from blending these steps together.
Technical commentary	This slide is about Warnock Algorithm. Read it as camera geometry. Track how 3D view-space positions become clip coordinates, normalized device coordinates, and finally screen locations. The visible cue is: - Hybrid object- and image-based divide-and-conquer approach / - Input: set of polygons and viewport / - Output: set of visible polygon parts / - Procedure: exploit spatial coherence of projected polygons to divide viewport / - Divide viewport recursively into squared regions
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Warnock Algorithm' changes positions before rasterization?
Page 33	Region-Based Visibility Determination
Source cue	■ ■ / Case 1 / - Given viewport region ■ and set of polygons ■, for the following / cases visibility determination can be considered trivial / - Case 1: All polygons in ■ are outside of ■ ■ ■ ■ ■ ■
Professor-style explanation	This slide should be read as camera geometry. With 'Region-Based Visibility Determination', the question is how a 3D view becomes coordinates that can be clipped, divided, mapped to the viewport, and rasterized. The slide gives us this anchor: ■ ■ / Case 1 / - Given viewport region ■ and set of polygons ■, for the following / cases visibility determination can be considered trivial / - Case 1: All polygons in ■ are outside of ■ ■ ■ ■ ■ ■. Keep separate the camera/view transform, the projection matrix, the perspective divide, and the final viewport transform; many mistakes come from blending these steps together.
Technical commentary	This slide is about Region-Based Visibility Determination. Read it as camera geometry. Track how 3D view-space positions become clip coordinates, normalized device coordinates, and finally screen locations. The visible cue is: ■ ■ / Case 1 / - Given viewport region ■ and set of polygons ■, for the following / cases visibility determination can be considered trivial / - Case 1: All polygons in ■ are outside of ■ ■ ■ ■ ■ ■
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Region-Based Visibility Determination' changes positions before rasterization?
Page 34	Procedure

Source cue	- Start with entire viewport region ■ / - Subdivide ■ into four equal subregions / - Terminate if subregion size matches pixel size / - Classify subregions based on trivial cases / - If subregion matches trivial cases: resolve
Professor-style explanation	This slide should be read as camera geometry. With 'Procedure', the question is how a 3D view becomes coordinates that can be clipped, divided, mapped to the viewport, and rasterized. The slide gives us this anchor: - Start with entire viewport region ■ / - Subdivide ■ into four equal subregions / - Terminate if subregion size matches pixel size / - Classify subregions based on trivial cases / - If subregion matches trivial cases: resolve. Keep separate the camera/view transform, the projection matrix, the perspective divide, and the final viewport transform; many mistakes come from blending these steps together.
Technical commentary	This slide is about Procedure. Read it as camera geometry. Track how 3D view-space positions become clip coordinates, normalized device coordinates, and finally screen locations. The visible cue is: - Start with entire viewport region ■ / - Subdivide ■ into four equal subregions / - Terminate if subregion size matches pixel size / - Classify subregions based on trivial cases / - If subregion matches trivial cases: resolve
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Procedure' changes positions before rasterization?

Page 35	Pseudo Code
Source cue	Procedure warnockHSR(Polygons list, Viewport vp) / If isSimple(list, vp) Then / // Base case: Simple enough scenario to draw directly / drawPolygons(list) / Else
Professor-style explanation	This slide should be read as camera geometry. With 'Pseudo Code', the question is how a 3D view becomes coordinates that can be clipped, divided, mapped to the viewport, and rasterized. The slide gives us this anchor: Procedure warnockHSR(Polygons list, Viewport vp) / If isSimple(list, vp) Then / // Base case: Simple enough scenario to draw directly / drawPolygons(list) / Else. Keep separate the camera/view transform, the projection matrix, the perspective divide, and the final viewport transform; many mistakes come from blending these steps together.
Technical commentary	This slide is about Pseudo Code. Read it as camera geometry. Track how 3D view-space positions become clip coordinates, normalized device coordinates, and finally screen locations. The visible cue is: Procedure warnockHSR(Polygons list, Viewport vp) / If isSimple(list, vp) Then / // Base case: Simple enough scenario to draw directly / drawPolygons(list) / Else
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Pseudo Code' changes positions before rasterization?

Page 36	Assessment Warnock Algorithm
---------	------------------------------

Source cue	- Advantages / - Efficient for large polygons by exploiting spatial coherence / - Easy to implement through recursive programming / - Disadvantages / - Restriction to polygons as primitives
Professor-style explanation	With 'Assessment Warnock Algorithm', the question is no longer just whether geometry exists, but whether it is visible from a viewpoint. The slide gives us this anchor: - Advantages / - Efficient for large polygons by exploiting spatial coherence / - Easy to implement through recursive programming / - Disadvantages / - Restriction to polygons as primitives. Explain the method by naming its decision space: does it compare objects, split image regions, cast rays, or compare per-fragment depth values? That tells you what it can handle well.
Technical commentary	This slide is about Assessment Warnock Algorithm. Read it as an occlusion decision. Decide whether the method reasons about objects, image regions, rays, or per-fragment depth comparisons. The visible cue is: - Advantages / - Efficient for large polygons by exploiting spatial coherence / - Easy to implement through recursive programming / - Disadvantages / - Restriction to polygons as primitives
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Assessment Warnock Algorithm' works per object, per image region, per ray, or per fragment?
Page 37	7.4 Depth Buffer Algorithm
Source cue	Exploiting image storage for visibility determination
Professor-style explanation	On this slide, I would emphasize that OpenGL is a controlled state machine around the GPU pipeline. '7.4 Depth Buffer Algorithm' is not about one magic render call; it is about objects, bindings, shader interfaces, buffers, and state being consistent at draw time. The slide gives us this anchor: Exploiting image storage for visibility determination. If the output is wrong, you inspect which object owns the data, which shader consumes it, and which state changes the result.
Technical commentary	This slide is about 7.4 Depth Buffer Algorithm. Read it as concrete API state and GPU data movement. Ask which object is bound, which shader stage consumes it, and which state affects the draw call. The visible cue is: Exploiting image storage for visibility determination
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in '7.4 Depth Buffer Algorithm'?
Page 38	Depth Buffer Algorithm 1/3
Source cue	- Image-based visibility determination algorithm / that exploits extra graphics memory called depth buffer (= z-buffer) / - Depth buffer / - Depth buffer is a 2D raster that contains depth values / - Depth value calculation happens during rasterization

Professor-style explanation	On this slide, I would emphasize that OpenGL is a controlled state machine around the GPU pipeline. 'Depth Buffer Algorithm 1/3' is not about one magic render call; it is about objects, bindings, shader interfaces, buffers, and state being consistent at draw time. The slide gives us this anchor: - Image-based visibility determination algorithm / that exploits extra graphics memory called depth buffer (= z-buffer) / - Depth buffer / - Depth buffer is a 2D raster that contains depth values / - Depth value calculation happens during rasterization. If the output is wrong, you inspect which object owns the data, which shader consumes it, and which state changes the result.
Technical commentary	This slide is about Depth Buffer Algorithm 1/3. Read it as concrete API state and GPU data movement. Ask which object is bound, which shader stage consumes it, and which state affects the draw call. The visible cue is: - Image-based visibility determination algorithm / that exploits extra graphics memory called depth buffer (= z-buffer) / - Depth buffer / - Depth buffer is a 2D raster that contains depth values / - Depth value calculation happens during rasterization
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Depth Buffer Algorithm 1/3'?

Page 39	Depth Buffer Algorithm 2/3
Source cue	- Depth buffer algorithm follows occlusion culling principle / Function <code>occlusionCulling(List of Polygons scene)</code> / Initialize an <code>OcclusionRepresentation OR</code> / For each object <code>obj</code> in scene / If <code>obj</code> is occluded by <code>OR</code> Then
Professor-style explanation	On this slide, I would emphasize that OpenGL is a controlled state machine around the GPU pipeline. 'Depth Buffer Algorithm 2/3' is not about one magic render call; it is about objects, bindings, shader interfaces, buffers, and state being consistent at draw time. The slide gives us this anchor: - Depth buffer algorithm follows occlusion culling principle / Function <code>occlusionCulling(List of Polygons scene)</code> / Initialize an <code>OcclusionRepresentation OR</code> / For each object <code>obj</code> in scene / If <code>obj</code> is occluded by <code>OR</code> Then. If the output is wrong, you inspect which object owns the data, which shader consumes it, and which state changes the result.
Technical commentary	This slide is about Depth Buffer Algorithm 2/3. Read it as concrete API state and GPU data movement. Ask which object is bound, which shader stage consumes it, and which state affects the draw call. The visible cue is: - Depth buffer algorithm follows occlusion culling principle / Function <code>occlusionCulling(List of Polygons scene)</code> / Initialize an <code>OcclusionRepresentation OR</code> / For each object <code>obj</code> in scene / If <code>obj</code> is occluded by <code>OR</code> Then
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Depth Buffer Algorithm 2/3'?

Page 40	Depth Buffer Algorithm 3/3
---------	----------------------------

Source cue	- Depth buffer algorithm performs depth testing for each fragment / - Depth buffer is initialized with background value (e.g., $z = 1$) / - For every scene object / - Rasterize scene object / - For each fragment
Professor-style explanation	On this slide, I would emphasize that OpenGL is a controlled state machine around the GPU pipeline. 'Depth Buffer Algorithm 3/3' is not about one magic render call; it is about objects, bindings, shader interfaces, buffers, and state being consistent at draw time. The slide gives us this anchor: - Depth buffer algorithm performs depth testing for each fragment / - Depth buffer is initialized with background value (e.g., $z = 1$) / - For every scene object / - Rasterize scene object / - For each fragment. If the output is wrong, you inspect which object owns the data, which shader consumes it, and which state changes the result.
Technical commentary	This slide is about Depth Buffer Algorithm 3/3. Read it as concrete API state and GPU data movement. Ask which object is bound, which shader stage consumes it, and which state affects the draw call. The visible cue is: - Depth buffer algorithm performs depth testing for each fragment / - Depth buffer is initialized with background value (e.g., $z = 1$) / - For every scene object / - Rasterize scene object / - For each fragment
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Depth Buffer Algorithm 3/3'?

Page 41	Pseudo Code – Depth Buffer Test
Source cue	Function zBufferHSR(List of Geometry list) // Initialize z-buffer and color buffer for each pixel on the screen / For x from 0 to Width / For y from 0 to Height / Set depthbuffer[x][y] to 1.0 // Maximum depth value
Professor-style explanation	On this slide, I would emphasize that OpenGL is a controlled state machine around the GPU pipeline. 'Pseudo Code – Depth Buffer Test' is not about one magic render call; it is about objects, bindings, shader interfaces, buffers, and state being consistent at draw time. The slide gives us this anchor: Function zBufferHSR(List of Geometry list) // Initialize z-buffer and color buffer for each pixel on the screen / For x from 0 to Width / For y from 0 to Height / Set depthbuffer[x][y] to 1.0 // Maximum depth value. If the output is wrong, you inspect which object owns the data, which shader consumes it, and which state changes the result.
Technical commentary	This slide is about Pseudo Code – Depth Buffer Test. Read it as concrete API state and GPU data movement. Ask which object is bound, which shader stage consumes it, and which state affects the draw call. The visible cue is: Function zBufferHSR(List of Geometry list) // Initialize z-buffer and color buffer for each pixel on the screen / For x from 0 to Width / For y from 0 to Height / Set depthbuffer[x][y] to 1.0 // Maximum depth value
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Pseudo Code – Depth Buffer Test'?

Page 42	OpenGL Depth Buffer
Source cue	- OpenGL depth buffer: initialization and activation / - Depth values are limited to [0, 1] (0: near-plane, 1: far-plane) / - Depth test must be explicitly activated / - Depth test comparison function can be changed / Function <code>init()</code>
Professor-style explanation	On this slide, I would emphasize that OpenGL is a controlled state machine around the GPU pipeline. 'OpenGL Depth Buffer' is not about one magic render call; it is about objects, bindings, shader interfaces, buffers, and state being consistent at draw time. The slide gives us this anchor: - OpenGL depth buffer: initialization and activation / - Depth values are limited to [0, 1] (0: near-plane, 1: far-plane) / - Depth test must be explicitly activated / - Depth test comparison function can be changed / Function <code>init()</code> . If the output is wrong, you inspect which object owns the data, which shader consumes it, and which state changes the result.
Technical commentary	This slide is about OpenGL Depth Buffer. Read it as concrete API state and GPU data movement. Ask which object is bound, which shader stage consumes it, and which state affects the draw call. The visible cue is: - OpenGL depth buffer: initialization and activation / - Depth values are limited to [0, 1] (0: near-plane, 1: far-plane) / - Depth test must be explicitly activated / - Depth test comparison function can be changed / Function <code>init()</code>
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Depth Buffer'?
Page 43	Depth Buffer Precision
Source cue	- Depth buffer precision is limited by its bit-depth (e.g., 16-bit, 24-bit, 32-bit), / which determines how finely depth can be distinguished / - Depth buffer precision is not linear: denser at near- and sparser at far- / clipping plane $\Rightarrow$ depth conflicts can occur especially at greater distances / Nvidia Blog
Professor-style explanation	On this slide, I would emphasize that OpenGL is a controlled state machine around the GPU pipeline. 'Depth Buffer Precision' is not about one magic render call; it is about objects, bindings, shader interfaces, buffers, and state being consistent at draw time. The slide gives us this anchor: - Depth buffer precision is limited by its bit-depth (e.g., 16-bit, 24-bit, 32-bit), / which determines how finely depth can be distinguished / - Depth buffer precision is not linear: denser at near- and sparser at far- / clipping plane $\Rightarrow$ depth conflicts can occur especially at greater distances / Nvidia Blog. If the output is wrong, you inspect which object owns the data, which shader consumes it, and which state changes the result.
Technical commentary	This slide is about Depth Buffer Precision. Read it as concrete API state and GPU data movement. Ask which object is bound, which shader stage consumes it, and which state affects the draw call. The visible cue is: - Depth buffer precision is limited by its bit-depth (e.g., 16-bit, 24-bit, 32-bit), / which determines how finely depth can be distinguished / - Depth buffer precision is not linear: denser at near- and sparser at far- / clipping plane $\Rightarrow$ depth conflicts can occur especially at greater distances / Nvidia Blog

Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Depth Buffer Precision'?
Page 44	Z-Fighting 1/2
Source cue	- Z-fighting occurs when two or more objects are very close together in depth / - Depth buffer can't consistently resolve which one is closer / causing flickering or overlapping artifacts in the rendered scene
Professor-style explanation	On this slide, I would emphasize that OpenGL is a controlled state machine around the GPU pipeline. 'Z-Fighting 1/2' is not about one magic render call; it is about objects, bindings, shader interfaces, buffers, and state being consistent at draw time. The slide gives us this anchor: - Z-fighting occurs when two or more objects are very close together in depth / - Depth buffer can't consistently resolve which one is closer / causing flickering or overlapping artifacts in the rendered scene. If the output is wrong, you inspect which object owns the data, which shader consumes it, and which state changes the result.
Technical commentary	This slide is about Z-Fighting 1/2. Read it as concrete API state and GPU data movement. Ask which object is bound, which shader stage consumes it, and which state affects the draw call. The visible cue is: - Z-fighting occurs when two or more objects are very close together in depth / - Depth buffer can't consistently resolve which one is closer / causing flickering or overlapping artifacts in the rendered scene
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Z-Fighting 1/2'?
Page 45	Z-Fighting 2/2
Source cue	- Mitigation through depth buffer properties / - Use higher precision buffers by switching from 24-bit to 32-bit depth / - Adjust near and far planes: bringing near plane out- and far plane inwards / reduces the depth range, improving effective depth buffer precision / - Mitigating through rendering properties
Professor-style explanation	On this slide, I would emphasize that OpenGL is a controlled state machine around the GPU pipeline. 'Z-Fighting 2/2' is not about one magic render call; it is about objects, bindings, shader interfaces, buffers, and state being consistent at draw time. The slide gives us this anchor: - Mitigation through depth buffer properties / - Use higher precision buffers by switching from 24-bit to 32-bit depth / - Adjust near and far planes: bringing near plane out- and far plane inwards / reduces the depth range, improving effective depth buffer precision / - Mitigating through rendering properties. If the output is wrong, you inspect which object owns the data, which shader consumes it, and which state changes the result.

Technical commentary	This slide is about Z-Fighting 2/2. Read it as concrete API state and GPU data movement. Ask which object is bound, which shader stage consumes it, and which state affects the draw call. The visible cue is: - Mitigation through depth buffer properties / - Use higher precision buffers by switching from 24-bit to 32-bit depth / - Adjust near and far planes: bringing near plane out- and far plane inwards / reduces the depth range, improving effective depth buffer precision / - Mitigating through rendering properties
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Z-Fighting 2/2'?

Page 46	Assessment Depth Buffering
Source cue	- Advantages / - Easy implementation / - No presorting of scene objects required / $\Rightarrow$ Object evaluation can be performed in any order / - Not limited to polygonal geometries
Professor-style explanation	On this slide, I would emphasize that OpenGL is a controlled state machine around the GPU pipeline. 'Assessment Depth Buffering' is not about one magic render call; it is about objects, bindings, shader interfaces, buffers, and state being consistent at draw time. The slide gives us this anchor: - Advantages / - Easy implementation / - No presorting of scene objects required / $\Rightarrow$ Object evaluation can be performed in any order / - Not limited to polygonal geometries. If the output is wrong, you inspect which object owns the data, which shader consumes it, and which state changes the result.
Technical commentary	This slide is about Assessment Depth Buffering. Read it as concrete API state and GPU data movement. Ask which object is bound, which shader stage consumes it, and which state affects the draw call. The visible cue is: - Advantages / - Easy implementation / - No presorting of scene objects required / $\Rightarrow$ Object evaluation can be performed in any order / - Not limited to polygonal geometries
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Assessment Depth Buffering'?

Page 47	7.5 Depth Buffer Extensions
Source cue	How to render semi-transparent scene objects, and how to generate halos
Professor-style explanation	On this slide, I would emphasize that OpenGL is a controlled state machine around the GPU pipeline. '7.5 Depth Buffer Extensions' is not about one magic render call; it is about objects, bindings, shader interfaces, buffers, and state being consistent at draw time. The slide gives us this anchor: How to render semi-transparent scene objects, and how to generate halos. If the output is wrong, you inspect which object owns the data, which shader consumes it, and which state changes the result.

Technical commentary	This slide is about 7.5 Depth Buffer Extensions. Read it as concrete API state and GPU data movement. Ask which object is bound, which shader stage consumes it, and which state affects the draw call. The visible cue is: How to render semi-transparent scene objects, and how to generate halos
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in '7.5 Depth Buffer Extensions'?

Page 48	Semi-Transparent Objects 1/2
Source cue	- Rendering of semi-transparent objects requires special handling / - Opaque and semi-transparent geometry need to be separated / - Opaque geometry is rendered first, semi-transparent blended on top / Function draw(Vector scene) / // Clear the framebuffer to prepare for new drawing
Professor-style explanation	On this slide, I would emphasize that OpenGL is a controlled state machine around the GPU pipeline. 'Semi-Transparent Objects 1/2' is not about one magic render call; it is about objects, bindings, shader interfaces, buffers, and state being consistent at draw time. The slide gives us this anchor: - Rendering of semi-transparent objects requires special handling / - Opaque and semi-transparent geometry need to be separated / - Opaque geometry is rendered first, semi-transparent blended on top / Function draw(Vector scene) / // Clear the framebuffer to prepare for new drawing. If the output is wrong, you inspect which object owns the data, which shader consumes it, and which state changes the result.
Technical commentary	This slide is about Semi-Transparent Objects 1/2. Read it as concrete API state and GPU data movement. Ask which object is bound, which shader stage consumes it, and which state affects the draw call. The visible cue is: - Rendering of semi-transparent objects requires special handling / - Opaque and semi-transparent geometry need to be separated / - Opaque geometry is rendered first, semi-transparent blended on top / Function draw(Vector scene) / // Clear the framebuffer to prepare for new drawing
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Semi-Transparent Objects 1/2'?

Page 49	Semi-Transparent Objects 2/2
Source cue	- Issue: correct transparency without presorting of scene objects not possible
Professor-style explanation	For 'Semi-Transparent Objects 2/2', I would not just read the bullet points aloud. I would ask what problem the slide is solving and how it connects to the previous and next stage. The slide gives us this anchor: - Issue: correct transparency without presorting of scene objects not possible. Turn the slide into a causal explanation: this input is processed by this idea, which produces this result, and that result matters later.

Technical commentary	This slide is about Semi-Transparent Objects 2/2. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - Issue: correct transparency without presorting of scene objects not possible
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Semi-Transparent Objects 2/2' into a causal sentence instead of repeating the slide title?

Page 50	Order-Independent Transparency 1/4
Source cue	- Order-independent transparency exploits depth peeling to avoid sorting / - Depth peeling peels away depth layers one-by-one in front-to-back order / - Extract nearest fragments with smallest z values in first rendering pass / - Extract next further fragments in each subsequent rendering pass / - The ■th rendering pass delivers the fragments to the nth depth plane
Professor-style explanation	With 'Order-Independent Transparency 1/4', the question is no longer just whether geometry exists, but whether it is visible from a viewpoint. The slide gives us this anchor: - Order-independent transparency exploits depth peeling to avoid sorting / - Depth peeling peels away depth layers one-by-one in front-to-back order / - Extract nearest fragments with smallest z values in first rendering pass / - Extract next further fragments in each subsequent rendering pass / - The ■th rendering pass delivers the fragments to the nth depth plane. Explain the method by naming its decision space: does it compare objects, split image regions, cast rays, or compare per-fragment depth values? That tells you what it can handle well.
Technical commentary	This slide is about Order-Independent Transparency 1/4. Read it as an occlusion decision. Decide whether the method reasons about objects, image regions, rays, or per-fragment depth comparisons. The visible cue is: - Order-independent transparency exploits depth peeling to avoid sorting / - Depth peeling peels away depth layers one-by-one in front-to-back order / - Extract nearest fragments with smallest z values in first rendering pass / - Extract next further fragments in each subsequent rendering pass / - The ■th rendering pass delivers the fragments to the nth depth plane
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Order-Independent Transparency 1/4' works per object, per image region, per ray, or per fragment?

Page 51	Order-Independent Transparency 2/4
Source cue	- Illustration of subsequent layers / Layer 0 Layer 1 Layer 2 / 0 depth 1 0 depth 1 0 depth 1

Professor-style explanation	With 'Order-Independent Transparency 2/4', the question is no longer just whether geometry exists, but whether it is visible from a viewpoint. The slide gives us this anchor: - Illustration of subsequent layers / Layer 0 Layer 1 Layer 2 / 0 depth 1 0 depth 1 0 depth 1. Explain the method by naming its decision space: does it compare objects, split image regions, cast rays, or compare per-fragment depth values? That tells you what it can handle well.
Technical commentary	This slide is about Order-Independent Transparency 2/4. Read it as an occlusion decision. Decide whether the method reasons about objects, image regions, rays, or per-fragment depth comparisons. The visible cue is: - Illustration of subsequent layers / Layer 0 Layer 1 Layer 2 / 0 depth 1 0 depth 1 0 depth 1
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Order-Independent Transparency 2/4' works per object, per image region, per ray, or per fragment?

Page 52	Order-Independent Transparency 3/4
Source cue	- 1st pass / - Render scene with conventional depth test (GL_LESS) / - Store resulting depth layer in Z-Buffer A / - Store resulting color layer as Layer 0 / - 2nd pass
Professor-style explanation	On this slide, I would emphasize that OpenGL is a controlled state machine around the GPU pipeline. 'Order-Independent Transparency 3/4' is not about one magic render call; it is about objects, bindings, shader interfaces, buffers, and state being consistent at draw time. The slide gives us this anchor: - 1st pass / - Render scene with conventional depth test (GL_LESS) / - Store resulting depth layer in Z-Buffer A / - Store resulting color layer as Layer 0 / - 2nd pass. If the output is wrong, you inspect which object owns the data, which shader consumes it, and which state changes the result.
Technical commentary	This slide is about Order-Independent Transparency 3/4. Read it as concrete API state and GPU data movement. Ask which object is bound, which shader stage consumes it, and which state affects the draw call. The visible cue is: - 1st pass / - Render scene with conventional depth test (GL_LESS) / - Store resulting depth layer in Z-Buffer A / - Store resulting color layer as Layer 0 / - 2nd pass
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Order-Independent Transparency 3/4'?

Page 53	Order-Independent Transparency 4/4
Source cue	1 layer 2 layers / 3 layers 4 layers
Professor-style explanation	For 'Order-Independent Transparency 4/4', I would not just read the bullet points aloud. I would ask what problem the slide is solving and how it connects to the previous and next stage. The slide gives us this anchor: 1 layer 2 layers / 3 layers 4 layers. Turn the slide into a causal explanation: this input is processed by this idea, which produces this result, and that result matters later.

Technical commentary	This slide is about Order-Independent Transparency 4/4. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: 1 layer 2 layers / 3 layers 4 layers
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Order-Independent Transparency 4/4' into a causal sentence instead of repeating the slide title?
Page 54	Generating Haloes 1/2
Source cue	- Halo: circle of light around sun or moon caused by ice crystals in the air; [...]; / a distinguishing zone surrounding a central object / - Haloes as wireframe illustration tools result / in better spatial comprehension / - Add highlights to foreground lines
Professor-style explanation	This slide belongs to local shading. For 'Generating Haloes 1/2', imagine one visible surface point and ask how bright or colored it should become. The slide gives us this anchor: - Halo: circle of light around sun or moon caused by ice crystals in the air; [...]; / a distinguishing zone surrounding a central object / - Haloes as wireframe illustration tools result / in better spatial comprehension / - Add highlights to foreground lines. The professor explanation must name the normal, light direction, view direction, material response, and whether the calculation is done per vertex or per fragment.
Technical commentary	This slide is about Generating Haloes 1/2. Read it as local shading. Identify normal, light direction, view direction, material coefficients, and where the computation is evaluated. The visible cue is: - Halo: circle of light around sun or moon caused by ice crystals in the air; [...]; / a distinguishing zone surrounding a central object / - Haloes as wireframe illustration tools result / in better spatial comprehension / - Add highlights to foreground lines
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Generating Haloes 1/2'?
Page 55	Generating Haloes 2/2
Source cue	- Proceeding / - Render wireframe model with thick lines into depth buffer / without writing to the color buffer $\Rightarrow$ depth image contains thick lines / - Render wireframe model with thin lines into color buffer / $\Rightarrow$ Lines of foreground surfaces have a "safety margin"

Professor-style explanation	On this slide, I would emphasize that OpenGL is a controlled state machine around the GPU pipeline. 'Generating Haloes 2/2' is not about one magic render call; it is about objects, bindings, shader interfaces, buffers, and state being consistent at draw time. The slide gives us this anchor: - Proceeding / - Render wireframe model with thick lines into depth buffer / without writing to the color buffer $\Rightarrow$ depth image contains thick lines / - Render wireframe model with thin lines into color buffer / $\Rightarrow$ Lines of foreground surfaces have a "safety margin". If the output is wrong, you inspect which object owns the data, which shader consumes it, and which state changes the result.
Technical commentary	This slide is about Generating Haloes 2/2. Read it as concrete API state and GPU data movement. Ask which object is bound, which shader stage consumes it, and which state affects the draw call. The visible cue is: - Proceeding / - Render wireframe model with thick lines into depth buffer / without writing to the color buffer $\Rightarrow$ depth image contains thick lines / - Render wireframe model with thin lines into color buffer / $\Rightarrow$ Lines of foreground surfaces have a "safety margin"
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Generating Haloes 2/2'?
Page 56	7.6 Ray Casting
Source cue	Shooting rays to determine visibility
Professor-style explanation	With '7.6 Ray Casting', the question is no longer just whether geometry exists, but whether it is visible from a viewpoint. The slide gives us this anchor: Shooting rays to determine visibility. Explain the method by naming its decision space: does it compare objects, split image regions, cast rays, or compare per-fragment depth values? That tells you what it can handle well.
Technical commentary	This slide is about 7.6 Ray Casting. Read it as an occlusion decision. Decide whether the method reasons about objects, image regions, rays, or per-fragment depth comparisons. The visible cue is: Shooting rays to determine visibility
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether '7.6 Ray Casting' works per object, per image region, per ray, or per fragment?
Page 57	Ray Casting Principle
Source cue	- Image-based visibility determination algorithm exploiting ray intersections / - Ray initialization - initialize ray from viewpoint through each pixel / (ray generation) / - Intersection calculation - find object intersection closest to camera / (ray intersection)

Professor-style explanation	This slide should be read as camera geometry. With 'Ray Casting Principle', the question is how a 3D view becomes coordinates that can be clipped, divided, mapped to the viewport, and rasterized. The slide gives us this anchor: - Image-based visibility determination algorithm exploiting ray intersections / - Ray initialization - initialize ray from viewpoint through each pixel / (ray generation) / - Intersection calculation - find object intersection closest to camera / (ray intersection). Keep separate the camera/view transform, the projection matrix, the perspective divide, and the final viewport transform; many mistakes come from blending these steps together.
Technical commentary	This slide is about Ray Casting Principle. Read it as camera geometry. Track how 3D view-space positions become clip coordinates, normalized device coordinates, and finally screen locations. The visible cue is: - Image-based visibility determination algorithm exploiting ray intersections / - Ray initialization - initialize ray from viewpoint through each pixel / (ray generation) / - Intersection calculation - find object intersection closest to camera / (ray intersection)
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Ray Casting Principle' changes positions before rasterization?
Page 58	Ray Initialization 1/3
Source cue	- Ray is defined by origin and direction / - Mathematically parameterized as a 3D line via $\mathbf{r} = \mathbf{o} + \mathbf{d}t$
Professor-style explanation	Here the lecture moves from continuous geometry to a discrete grid. 'Ray Initialization 1/3' asks which pixels or samples are covered by an ideal mathematical primitive. The slide gives us this anchor: - Ray is defined by origin and direction / - Mathematically parameterized as a 3D line via $\mathbf{r} = \mathbf{o} + \mathbf{d}t$. The key spoken explanation is: rasterization creates fragment candidates and interpolated values, but it does not by itself guarantee that a fragment becomes the final visible pixel.
Technical commentary	This slide is about Ray Initialization 1/3. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: - Ray is defined by origin and direction / - Mathematically parameterized as a 3D line via $\mathbf{r} = \mathbf{o} + \mathbf{d}t$
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Ray Initialization 1/3'?
Page 59	Ray Initialization 2/3
Source cue	- How can we find $\mathbf{t}$? / - Assumption: we have a simplified camera model / - Positioned in $\mathbf{o}$ / - Orthonormal base $\{\mathbf{u}, \mathbf{v}, \mathbf{w}\}$ / - Orthonormal: orthogonal + normalized

Professor-style explanation	This slide should be read as camera geometry. With 'Ray Initialization 2/3', the question is how a 3D view becomes coordinates that can be clipped, divided, mapped to the viewport, and rasterized. The slide gives us this anchor: - How can we find $\mathbf{p}$? / - Assumption: we have a simplified camera model / - Positioned in $\mathbf{p}$ / - Orthonormal base $\{\mathbf{u}, \mathbf{v}, \mathbf{w}\}$ / - Orthonormal: orthogonal + normalized. Keep separate the camera/view transform, the projection matrix, the perspective divide, and the final viewport transform; many mistakes come from blending these steps together.
Technical commentary	This slide is about Ray Initialization 2/3. Read it as camera geometry. Track how 3D view-space positions become clip coordinates, normalized device coordinates, and finally screen locations. The visible cue is: - How can we find $\mathbf{p}$? / - Assumption: we have a simplified camera model / - Positioned in $\mathbf{p}$ / - Orthonormal base $\{\mathbf{u}, \mathbf{v}, \mathbf{w}\}$ / - Orthonormal: orthogonal + normalized
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Ray Initialization 2/3' changes positions before rasterization?

Page 60	Ray Initialization 3/3
Source cue	- Definition of the pixel grid with $\mathbf{u} \times \mathbf{v}$ pixels / x y / - Size of the pixel raster is $\mathbf{u} - \mathbf{u} \times \mathbf{v} - \mathbf{v}$ / - Pixel spacing is / - x-coordinate: $(\mathbf{u} - \mathbf{u})/\mathbf{u}$
Professor-style explanation	For 'Ray Initialization 3/3', put the formula aside for a moment and name the coordinate spaces. A transformation only makes sense when we know where the point, vector, or normal starts and where it should end. The slide gives us this anchor: - Definition of the pixel grid with $\mathbf{u} \times \mathbf{v}$ pixels / x y / - Size of the pixel raster is $\mathbf{u} - \mathbf{u} \times \mathbf{v} - \mathbf{v}$ / - Pixel spacing is / - x-coordinate: $(\mathbf{u} - \mathbf{u})/\mathbf{u}$. The professor-level way to read this slide is to narrate the movement: model space to world space, world to view, view to clip, or whichever step the slide is showing.
Technical commentary	This slide is about Ray Initialization 3/3. Read it as a coordinate-space operation. Name the input space, the matrix or transformation, and the output space before memorizing formulas. The visible cue is: - Definition of the pixel grid with $\mathbf{u} \times \mathbf{v}$ pixels / x y / - Size of the pixel raster is $\mathbf{u} - \mathbf{u} \times \mathbf{v} - \mathbf{v}$ / - Pixel spacing is / - x-coordinate: $(\mathbf{u} - \mathbf{u})/\mathbf{u}$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Ray Initialization 3/3'?

Page 61	Intersection Calculation
Source cue	- Find smallest positive $\mathbf{t}$ for which $\mathbf{p}(\mathbf{t})$ is an intersection / - Simple procedure / - Iterate over all scene objects and calculate intersections / - Sort intersections / - Select intersection with smallest $\mathbf{t}$

Professor-style explanation	For 'Intersection Calculation', I would not just read the bullet points aloud. I would ask what problem the slide is solving and how it connects to the previous and next stage. The slide gives us this anchor: - Find smallest positive t for which $\mathbf{p} + t(\mathbf{d})$ is an intersection / - Simple procedure / - Iterate over all scene objects and calculate intersections / - Sort intersections / - Select intersection with smallest t . Turn the slide into a causal explanation: this input is processed by this idea, which produces this result, and that result matters later.
Technical commentary	This slide is about Intersection Calculation. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - Find smallest positive t for which $\mathbf{p} + t(\mathbf{d})$ is an intersection / - Simple procedure / - Iterate over all scene objects and calculate intersections / - Sort intersections / - Select intersection with smallest t
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Intersection Calculation' into a causal sentence instead of repeating the slide title?

Page 62	Implicit Surface Intersection Calculation
Source cue	- Find set of all points with $\mathbf{p} \cdot \mathbf{n} = 0$ / - Examples: plane, sphere, quadrics, ... / $\mathbf{p} = (x, y, z)$ / $\mathbf{n} = (n_x, n_y, n_z)$ / $\mathbf{p} \cdot \mathbf{n} = x n_x + y n_y + z n_z$
Professor-style explanation	For 'Implicit Surface Intersection Calculation', I would not just read the bullet points aloud. I would ask what problem the slide is solving and how it connects to the previous and next stage. The slide gives us this anchor: - Find set of all points with $\mathbf{p} \cdot \mathbf{n} = 0$ / - Examples: plane, sphere, quadrics, ... / $\mathbf{p} = (x, y, z)$ / $\mathbf{n} = (n_x, n_y, n_z)$ / $\mathbf{p} \cdot \mathbf{n} = x n_x + y n_y + z n_z$. Turn the slide into a causal explanation: this input is processed by this idea, which produces this result, and that result matters later.
Technical commentary	This slide is about Implicit Surface Intersection Calculation. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - Find set of all points with $\mathbf{p} \cdot \mathbf{n} = 0$ / - Examples: plane, sphere, quadrics, ... / $\mathbf{p} = (x, y, z)$ / $\mathbf{n} = (n_x, n_y, n_z)$ / $\mathbf{p} \cdot \mathbf{n} = x n_x + y n_y + z n_z$
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Implicit Surface Intersection Calculation' into a causal sentence instead of repeating the slide title?

Page 63	Intersection Calculation - Triangle
Source cue	- Idea: find intersection with plane which embeds triangle, / and test whether intersection lies inside triangle / 1. Calculate intersection with plane / through implicit surface intersection calculation / 2. Exploit barycentric coordinates

Professor-style explanation	For 'Intersection Calculation - Triangle', put the formula aside for a moment and name the coordinate spaces. A transformation only makes sense when we know where the point, vector, or normal starts and where it should end. The slide gives us this anchor: - Idea: find intersection with plane which embeds triangle, / and test whether intersection lies inside triangle / 1. Calculate intersection with plane / through implicit surface intersection calculation / 2. Exploit barycentric coordinates. The professor-level way to read this slide is to narrate the movement: model space to world space, world to view, view to clip, or whichever step the slide is showing.
Technical commentary	This slide is about Intersection Calculation - Triangle. Read it as a coordinate-space operation. Name the input space, the matrix or transformation, and the output space before memorizing formulas. The visible cue is: - Idea: find intersection with plane which embeds triangle, / and test whether intersection lies inside triangle / 1. Calculate intersection with plane / through implicit surface intersection calculation / 2. Exploit barycentric coordinates
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Intersection Calculation - Triangle'?
Page 64	Pseudo Code – Analytic Intersection Computation
Source cue	Function castRay(List sceneObjects, Window win, Camera cam) // Iterate over each pixel on the window / For x from 0 to win.width / For y from 0 to win.height / Initialize ray r = calculateRay(x, y, win, cam) // Calculate ray from camera through pixel (x, y)
Professor-style explanation	This slide should be read as camera geometry. With 'Pseudo Code – Analytic Intersection Computation', the question is how a 3D view becomes coordinates that can be clipped, divided, mapped to the viewport, and rasterized. The slide gives us this anchor: Function castRay(List sceneObjects, Window win, Camera cam) // Iterate over each pixel on the window / For x from 0 to win.width / For y from 0 to win.height / Initialize ray r = calculateRay(x, y, win, cam) // Calculate ray from camera through pixel (x, y). Keep separate the camera/view transform, the projection matrix, the perspective divide, and the final viewport transform; many mistakes come from blending these steps together.
Technical commentary	This slide is about Pseudo Code – Analytic Intersection Computation. Read it as camera geometry. Track how 3D view-space positions become clip coordinates, normalized device coordinates, and finally screen locations. The visible cue is: Function castRay(List sceneObjects, Window win, Camera cam) // Iterate over each pixel on the window / For x from 0 to win.width / For y from 0 to win.height / Initialize ray r = calculateRay(x, y, win, cam) // Calculate ray from camera through pixel (x, y)
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Pseudo Code – Analytic Intersection Computation' changes positions before rasterization?

Page 65	Pseudo Code - Iterative Intersection Calculation (Ray Marching)
Source cue	- Ray march in ■ steps and check all scene objects for intersection / Function intersect(const Ray& ray, const List of SceneObjects& sceneObjects) const / Initialize t as 0.0 // Starting point for ray parameter / Initialize epsilon as 0.01 // Small increment for ray parameter / // Loop to incrementally trace the ray through the scene
Professor-style explanation	With 'Pseudo Code - Iterative Intersection Calculation (Ray Marching)', the question is no longer just whether geometry exists, but whether it is visible from a viewpoint. The slide gives us this anchor: - Ray march in ■ steps and check all scene objects for intersection / Function intersect(const Ray& ray, const List of SceneObjects& sceneObjects) const / Initialize t as 0.0 // Starting point for ray parameter / Initialize epsilon as 0.01 // Small increment for ray parameter / // Loop to incrementally trace the ray through the scene. Explain the method by naming its decision space: does it compare objects, split image regions, cast rays, or compare per-fragment depth values? That tells you what it can handle well.
Technical commentary	This slide is about Pseudo Code - Iterative Intersection Calculation (Ray Marching). Read it as an occlusion decision. Decide whether the method reasons about objects, image regions, rays, or per-fragment depth comparisons. The visible cue is: - Ray march in ■ steps and check all scene objects for intersection / Function intersect(const Ray& ray, const List of SceneObjects& sceneObjects) const / Initialize t as 0.0 // Starting point for ray parameter / Initialize epsilon as 0.01 // Small increment for ray parameter / // Loop to incrementally trace the ray through the scene
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Pseudo Code - Iterative Intersection Calculation (Ray Marching)' works per object, per image region, per ray, or per fragment?
Page 66	Assessment Ray Casting
Source cue	- Advantages / - Image-precise algorithm that support all types of scene objects / - Analytic intersection calculation possible / - Can be parallelized and supported by graphics hardware / - Disadvantages
Professor-style explanation	With 'Assessment Ray Casting', the question is no longer just whether geometry exists, but whether it is visible from a viewpoint. The slide gives us this anchor: - Advantages / - Image-precise algorithm that support all types of scene objects / - Analytic intersection calculation possible / - Can be parallelized and supported by graphics hardware / - Disadvantages. Explain the method by naming its decision space: does it compare objects, split image regions, cast rays, or compare per-fragment depth values? That tells you what it can handle well.
Technical commentary	This slide is about Assessment Ray Casting. Read it as an occlusion decision. Decide whether the method reasons about objects, image regions, rays, or per-fragment depth comparisons. The visible cue is: - Advantages / - Image-precise algorithm that support all types of scene objects / - Analytic intersection calculation possible / - Can be parallelized and supported by graphics hardware / - Disadvantages

Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Assessment Ray Casting' works per object, per image region, per ray, or per fragment?
Page 67	but are crucial for accurate visibility determination
Source cue	- Visibility determination ensures only scene objects visible from camera are rendered / - Effective visibility determination is crucial for rendering efficiency and visual accuracy / - Object-based algorithms perform geometry-based analysis prior to rasterization / - Examples include the Painter's Algorithm, Weiler-Atherton, and Back Face Culling / - These methods often suffer from higher complexity and can be poorly parallelizable
Professor-style explanation	This slide should be read as camera geometry. With 'but are crucial for accurate visibility determination', the question is how a 3D view becomes coordinates that can be clipped, divided, mapped to the viewport, and rasterized. The slide gives us this anchor: - Visibility determination ensures only scene objects visible from camera are rendered / - Effective visibility determination is crucial for rendering efficiency and visual accuracy / - Object-based algorithms perform geometry-based analysis prior to rasterization / - Examples include the Painter's Algorithm, Weiler-Atherton, and Back Face Culling / - These methods often suffer from higher complexity and can be poorly parallelizable. Keep separate the camera/view transform, the projection matrix, the perspective divide, and the final viewport transform; many mistakes come from blending these steps together.
Technical commentary	This slide is about but are crucial for accurate visibility determination. Read it as camera geometry. Track how 3D view-space positions become clip coordinates, normalized device coordinates, and finally screen locations. The visible cue is: - Visibility determination ensures only scene objects visible from camera are rendered / - Effective visibility determination is crucial for rendering efficiency and visual accuracy / - Object-based algorithms perform geometry-based analysis prior to rasterization / - Examples include the Painter's Algorithm, Weiler-Atherton, and Back Face Culling / - These methods often suffer from higher complexity and can be poorly parallelizable
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'but are crucial for accurate visibility determination' changes positions before rasterization?
Page 68	Literature and other sources used in this chapter
Source cue	The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image.

Professor-style explanation	For 'Literature and other sources used in this chapter', I would not just read the bullet points aloud. I would ask what problem the slide is solving and how it connects to the previous and next stage. The slide gives us this anchor: The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image. Turn the slide into a causal explanation: this input is processed by this idea, which produces this result, and that result matters later.
Technical commentary	This slide is about Literature and other sources used in this chapter. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Literature and other sources used in this chapter' into a causal sentence instead of repeating the slide title?

Page 69	Practice, Addison-Wesley.
Source cue	- Text Books / - Foley J., van Dam, A., Feiner, S. (2013). Computer Graphics: Principles and / Practice, Addison-Wesley. / - Papers / - Everitt, C. (2001). Interactive order-independent transparency. White paper,
Professor-style explanation	For 'Practice, Addison-Wesley.', I would not just read the bullet points aloud. I would ask what problem the slide is solving and how it connects to the previous and next stage. The slide gives us this anchor: - Text Books / - Foley J., van Dam, A., Feiner, S. (2013). Computer Graphics: Principles and / Practice, Addison-Wesley. / - Papers / - Everitt, C. (2001). Interactive order-independent transparency. White paper,. Turn the slide into a causal explanation: this input is processed by this idea, which produces this result, and that result matters later.
Technical commentary	This slide is about Practice, Addison-Wesley.. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - Text Books / - Foley J., van Dam, A., Feiner, S. (2013). Computer Graphics: Principles and / Practice, Addison-Wesley. / - Papers / - Everitt, C. (2001). Interactive order-independent transparency. White paper,
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Practice, Addison-Wesley.' into a causal sentence instead of repeating the slide title?

07.1 Object-Based Algorithms

Source location: Section 7.1

Commentary

Object-based visibility algorithms reason about geometry before or while comparing surfaces. They can sort, split, or reject objects based on spatial relations. This is different from simply letting every fragment fight in the depth buffer.

The value of object-based reasoning is reducing work or establishing correct order. The difficulty is that geometry can overlap in complicated ways, so simple global ordering is not always possible.

Mental model

Object-based visibility tries to solve visibility with geometry before final pixels are written.

Check yourself

Why can intersecting polygons make simple depth sorting fail?

07.2 Binary Space Partitioning

Source location: Section 7.2

Commentary

A BSP tree recursively divides space with planes. Once built, it can help traverse geometry in a view-dependent order. This is useful for visibility and ordering because the tree encodes spatial relationships.

The cost is preprocessing and possible splitting of geometry. BSP is a good example of trading memory and setup work for faster or more structured visibility decisions later.

Mental model

A BSP tree stores a recursive answer to 'which side of this plane is the geometry on?'

Check yourself

Why can building a BSP tree require splitting polygons?

07.3 Warnock Algorithm

Source location: Section 7.3

Commentary

Warnock's algorithm works in image space by subdividing regions until visibility is simple enough to decide. If a region is ambiguous, split it. If it is simple, fill it.

This illustrates a general graphics strategy: recursively reduce a hard global problem into smaller local problems. The algorithm is less central in modern OpenGL practice than the depth buffer, but it helps contrast image-space and object-space approaches.

Mental model

When a region is too complicated, subdivide until the answer becomes simple.

Check yourself

What makes Warnock's algorithm image-space rather than object-space?

07.4 Depth Buffer Algorithm

Source location: Section 7.4

Commentary

The depth buffer stores the closest accepted depth at each sample or pixel. For each fragment, compare its depth to the stored depth. If it passes, update the color and depth; otherwise discard it.

The strength of the depth buffer is simplicity and hardware efficiency. The weakness is precision: depth values are finite, and projection distributes precision unevenly. Bad near/far settings can create z-fighting.

Mental model

The depth buffer is a per-sample competition for closest visible fragment.

Check yourself

Why should the near plane not be unnecessarily close to the camera?

07.5 Depth Extensions and Ray Casting

Source location: Sections 7.5-7.6

Commentary

Depth buffer extensions refine or adapt depth-based visibility, for example by handling precision, transparency, or multiple layers more carefully. Standard depth testing alone is not a complete solution for every visibility situation.

Ray casting takes the opposite direction from rasterization: for each image sample, cast a ray into the scene and find the closest intersection. This naturally solves primary visibility but requires intersection work instead of raster coverage work.

Mental model

Rasterization pushes primitives to pixels; ray casting pulls visibility from pixels into the scene.

Check yourself

Why is transparency harder than opaque nearest-surface visibility?

End-of-lecture summary

If you remember only one thing from Lecture 07, remember this: This lecture asks which surfaces are visible from a viewpoint. Rasterization can generate many fragment candidates, but visibility decides which ones should affect the image.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.